



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа №4

**Выполнение задач обработки данных
в распределённых вычислениях**

Студент группы ИНБО-12-23, Губарев С.А.

(подпись)

Преподаватель Воронцов Ю.А.

(подпись)

Москва 2026 г.

Содержание

1 Цель и условия эксперимента	3
2 Стенд и исходные данные	3
3 Ленивые вычисления и партиции	4
4 Repartition и coalesce	4
5 GroupBy и физические операторы	5
6 Сравнение стратегий JOIN	6
7 Повторное вычисление и persist	8
8 Benchmark и маленькие файлы	10
9 Журнал Spark UI	11
10 Самостоятельный вариант В	12
Самостоятельные выводы по варианту В	16
11 Выводы и ограничения	17
Ответы на контрольные вопросы	18
Источники	23

1 Цель и условия эксперимента

Цель работы — сопоставить логический результат Spark-обработки с физическим планом, стадиями, задачами, shuffle и затратами времени. Исследованы lazy evaluation, repartition/coalesce, groupBy, две стратегии JOIN, persist, число партиций и выходных файлов. Самостоятельная часть выполнена по варианту В: order_items и sellers, группировка по seller_state.

На данном стенде Broadcast Hash Join оказался быстрее Sort Merge Join в трёх прогретых повторениях. Persist ускорил повторные действия, но с учётом материализации не окупился за два действия. В коротком синтетическом benchmark увеличение числа партиций не дало ускорения; эти результаты относятся только к локальной VM.

2 Стенд и исходные данные

Результаты приведены в таблице 1.

Таблица 1 – Паспорт стенда

Параметр	Фактическое значение
Среда	VirtualBox, Ubuntu 22.04.5
Ресурсы VM	2 vCPU, 4 ГБ ОЗУ, диск 40 ГБ
Процессор хоста	AMD Ryzen 7 9700X
Python	3.10.12
Java	openjdk version "17.0.20.1" 2026-08-18
Spark	3.5.7
Master и parallelism	local[*], 2
Shuffle partitions	8
AQE	false

Python 3.10.12 используется из изолированного окружения Ubuntu. Методичка рекомендует Python 3.11, но выбранная версия совместима с Spark 3.5.7 и JDK 17. Local[*] использует два доступных виртуальных ядра и не является физическим кластером.

Во всей основной серии фиксированы AQE=false, spark.sql.shuffle.partitions=8 и autoBroadcastJoinThreshold=-1. Driver memory — 1536 МБ. Работают фоновые службы учебного стенда второй практики; их конфигурация не менялась, но фоновая нагрузка остаётся ограничением.

SQLite восстановлена из Olist второй практики; подготовка CSV и Parquet не входит в измеряемые действия.

Результаты приведены в таблице 2.

Таблица 2 – Входные DataFrame Parquet

Таблица	Строки	Партиции
geolocation	1 000 163	2
order_items	112 650	2
products	32 951	1
sellers	3 095	1
product_category_name_translation	71	1

3 Ленивые вычисления и партиции

В группе P4_LAZY до построения select/filter/withColumn и после него список Jobs пуст. После count появился Job 26, получено 1 000 163 строк за 0,166 с. Count требует подсчёта строк, но Catalyst может исключить неиспользуемое abs_lat; это не замер стоимости вычисления всех признаков.

Две входные партиции geolocation не совпадают с восемью SQL shuffle partitions. Структура «действие → Jobs → Stages → Tasks» зависит от физического плана: глобальная агрегация count включает финальную задачу даже без предметной группировки.

Результаты приведены в таблице 3.

Таблица 3 – Пять наиболее представленных штатов geolocation

Штат	Строки	Доля %
SP	404 268	40,42
MG	126 336	12,63
RJ	121 169	12,11
RS	61 851	6,18
PR	57 859	5,78

4 Repartition и coalesce

Гео сначала перераспределена в 16 разделов по штату, затем уменьшена до четырёх двумя способами. В плане repartition(4) оптимизатор свёл последовательные перераспределения к Exchange RoundRobinPartitioning(4). План coalesce(4) содержит Coalesce над исходным Exchange hashpartitioning(state, 16): нового полного shuffle нет, но исходный обмен

сохраняется. Поэтому отсутствие дополнительного Exchange не означает отсутствия shuffle во всём плане.

Результаты приведены в таблице 4.

Таблица 4 – Распределение строк по четырём выходным партициям

Способ	Минимум	Максимум	Среднее
repartition(4)	250 040	250 041	250 040,75
coalesce(4)	88 462	626 485	250 040,75

Repartition выровнял размеры до 250 040–250 041 строки, coalesce сохранил диапазон 88 462–626 485. Замеры count составили 0,309 и 0,206 с соответственно. Единичные времена иллюстративны; механизм подтверждён планом и UI, а не только скоростью.

На рисунке 1 показан результат выполнения на стенде.

```
TIABD PRACTICE 4 | PARTITIONS
variant      min      max      mean
skewed16     1301    466119   83346,92
repartition4 250040   250041  250040,75
coalesce4     88462    626485  250040,75
Lazy: {'before': [], 'after_transform': [], 'after_action': [26],
'rows': 1000163, 'seconds': 0.16576342700045643}
student@tiabd-pr2:~/spark-lab$
```

Рисунок 1 – Фактические размеры партиций и появление Job после action

5 GroupBy и физические операторы

Группировка по seller_id содержит частичную HashAggregate, Exchange hashpartitioning(seller_id, 8) и финальную HashAggregate. В Job 39 выполнены стадии 73 и 74 с 2 и 8 задачами; объём Shuffle Read и Write — по 269 271 байту.

Это реальный обмен промежуточными агрегатами, а не отправка всех исходных столбцов.

Результаты приведены в таблице 5.

Таблица 5 – Пять продавцов с большой суммой price

Продавец	Позиции	Сумма price	Среднее
4869f7a5dfa2...	1156	229 472,63	198,51
53243585a1d6...	410	222 776,05	543,36
4a3ca9315b74...	1987	200 472,92	100,89
fa1c13f2614d...	586	194 042,03	331,13
7c67e1448b00...	1364	187 923,89	137,77

План примера price ≥ 100 читается от Scan parquet к Filter, Project, частичной HashAggregate, Exchange и финальной HashAggregate. Отдельный orderBy добавляет Sort. Номера узлов и внутренних атрибутов не являются устойчивым интерфейсом.

6 Сравнение стратегий JOIN

Обе стратегии соединяют order_items с products по product_id и возвращают 112 650 строк. Sort Merge Join имеет Exchange и Sort с обеих сторон; Broadcast Hash Join передаёт правый справочник через BroadcastExchange и не делает key-shuffle большой стороны. При count остаётся небольшой shuffle финального счётчика: BHJ не означает ноль любого обмена.

Результаты приведены в таблице 6.

Таблица 6 – Измерения JOIN

Стратегия	Первый count c	Медиана 3 повторов c	Строки
SMJ	0,343	0,182	112 650
BHJ	0,217	0,111	112 650

Медианы SMJ/BHJ: 0,182 / 0,111 с. По первому измеренному action SMJ получил 4 стадии и 12 задач с 3 917 308 байт shuffle, BHJ — 3 стадии в двух Jobs и 4 задачи с 118 байт shuffle. Broadcast имеет собственные расходы и память, не включённые в счётчик shuffle.

На рисунке 2 показан результат выполнения на стенде.

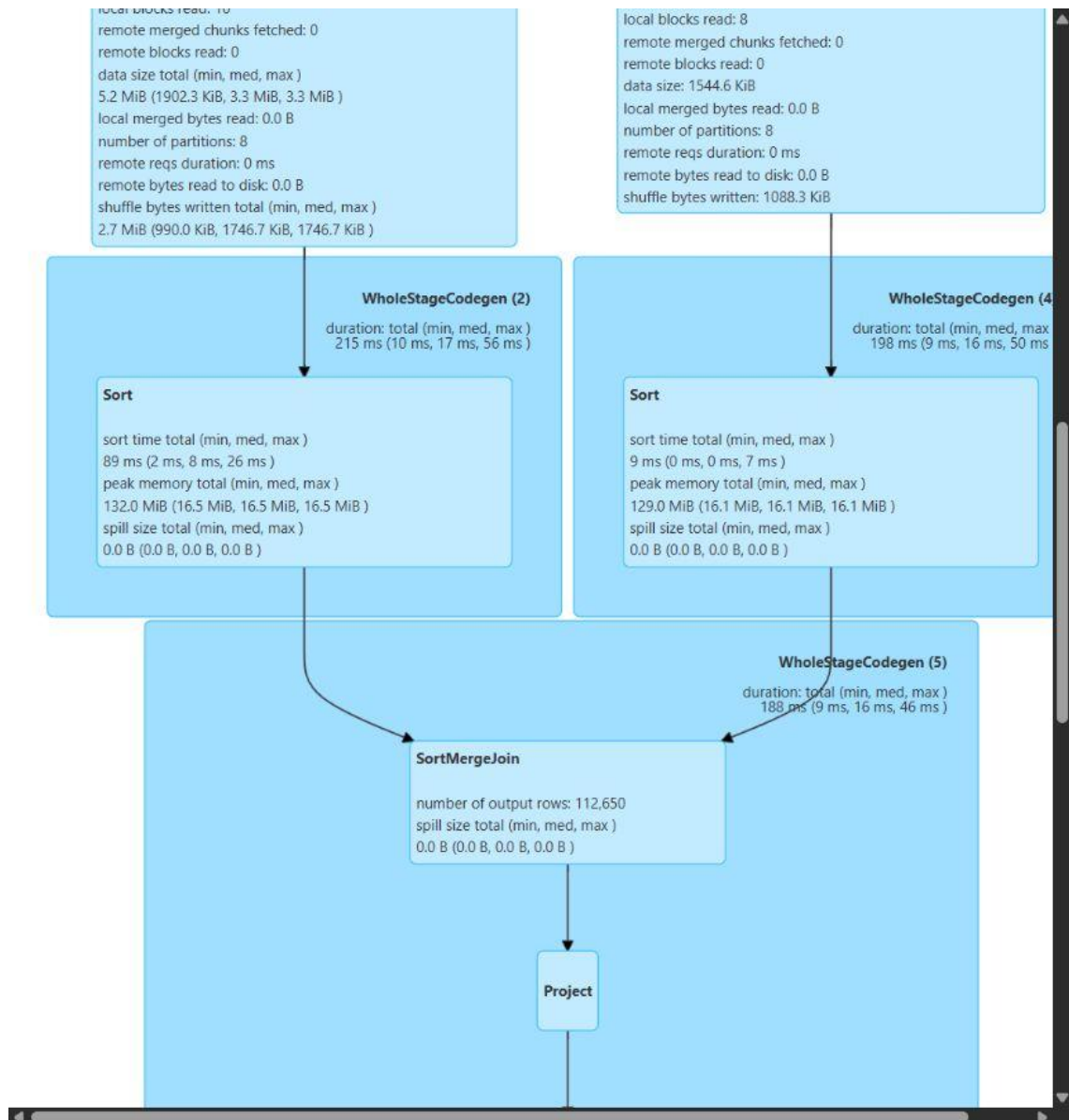


Рисунок 2 – Реальные Sort и SortMergeJoin в SQL DataFrame UI

На рисунке 3 показан результат выполнения на стенде.

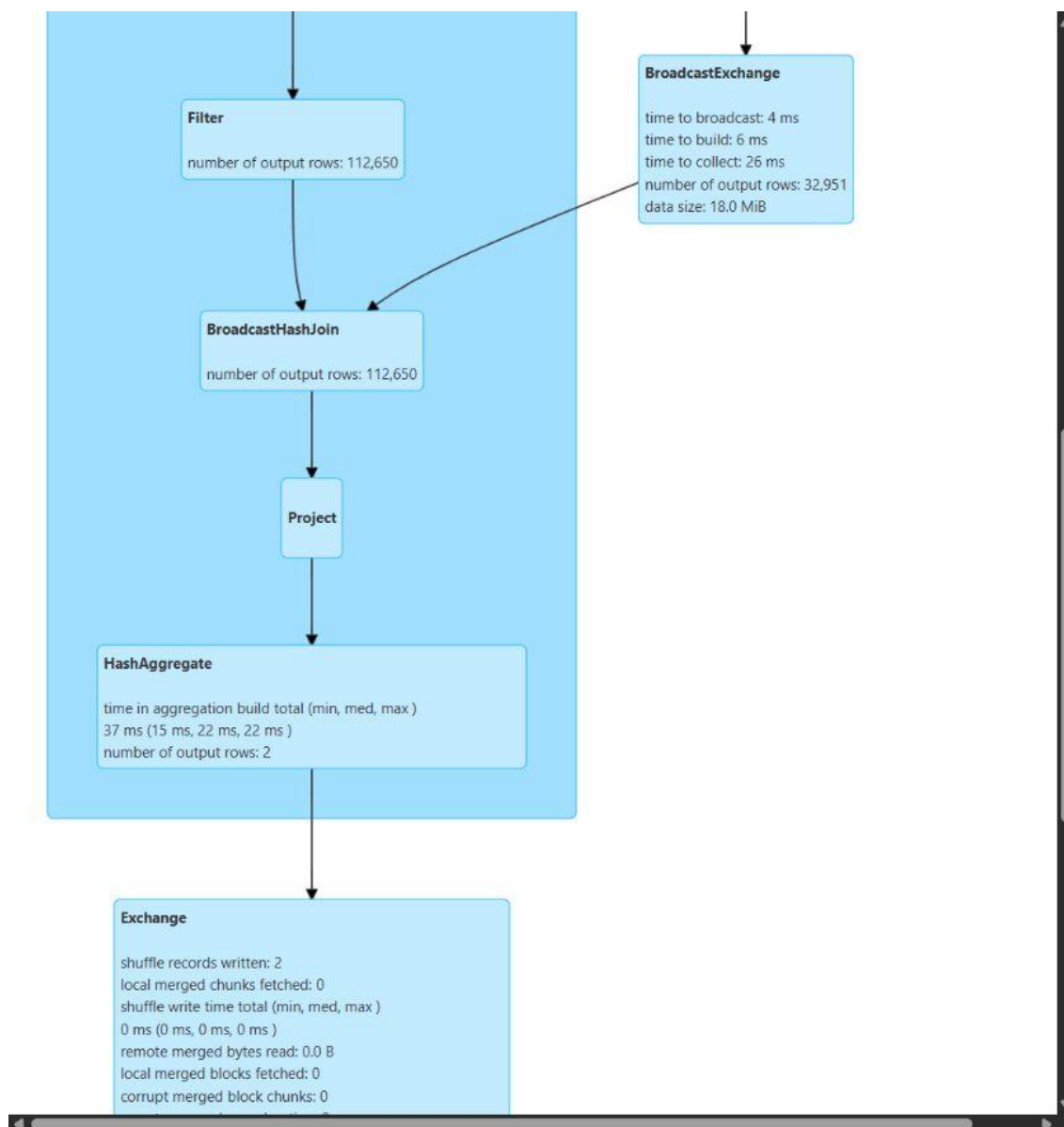


Рисунок 3 – Реальные BroadcastExchange и BroadcastHashJoin

7 Повторное вычисление и persist

Промежуточный DataFrame соединяет позиции с товарами и продавцами и вычисляет `gross_value`. Два действия агрегируют его по категории и штату продавца, затем возвращают контрольные суммы. `Persist(MEMORY_AND_DISK)` материализован count; суммы до и после кэша совпадают с допуском 0,01.

Результаты приведены в таблице 7.

Таблица 7 – Полная стоимость двух действий

Вариант	Заполнение кэша, с	Действие 1 с	Действие 2 с	Итого с
Без кэша	—	0,279	0,222	0,502
С persist	0,531	0,136	0,058	0,726

Повторные действия занимают 0,195 с вместо 0,502 с, но материализация стоит 0,531 с. Общая стоимость с кэшем 0,726 с выше исходной: при двух действиях сохранение не окупилось. UI показывает две сохранённые партии, 100% материализации и около 8,3 МиБ в памяти. После unpersist(blocking=True) is_cached=False.

На рисунке 4 показан результат выполнения на стенде.

The screenshot shows the Apache Spark 3.5.7 Storage tab. The 'RDDs' section is expanded, showing a single RDD with ID 476. The 'Storage Level' is 'Disk Memory Serialized 1x Replicated'. The 'Cached Partitions' is 2, and the 'Fraction Cached' is 100.00%.

ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached
476	*(3) Project [seller_id#206, product_id#205, order_id#203, order_item_id#204, shipping_limit_date#207, price#208, freight_value#209, product_category_name#218, seller_state#238, (price#208 + freight_value#209) AS gross_value#710] +- *(3) BroadcastHashJoin [seller_id#206], [seller_id#235], LeftOuter, BuildRight, false :- *(3) Project [product_id#205, order_id#203, order_item_id#204, seller_id#206, shipping_limit_date#207, price#208, freight_value#209, product_category_name#218] : +- *(3) BroadcastHashJoin [product_id#205], [product_id#217], LeftOuter, BuildRight, false :- *(3) ColumnarToRow : +- FileScan parquet [order_id#203,order_item_id#204,product_id#205,seller_id#206,shipping_limit_date#207,price#208,freight_value#209] Batched: true, DataFilters: [], Format: Parquet, Location: InMemoryFileIndex(1 paths)[file:/home/student/spark-lab/data/spark_input_parquet/order_items], PartitionFilters: [], PushedFilters: [], ReadSchema: struct<order_id:string,order_item_id:int,product_id:str...	Disk Memory Serialized 1x Replicated	2	100.00%

Рисунок 4 – Подтверждение материализации промежуточного DataFrame в Storage

8 Benchmark и маленькие файлы

Синтетический набор имеет $N=2\,000\,000$, ключ $id \% 1000$ и $value=\sqrt{id+1}$. После прогрева выполнены три запуска для каждого $p=1,2,4,8,16,32$. Shuffle partitions равны 8. Контрольные суммы близки к 1 885 618 790,063; небольшое различие последних знаков обусловлено порядком сложения double.

Результаты приведены в таблице 8.

Таблица 8 – Медианы синтетического benchmark

p	t1 с	t2 с	t3 с	Медиана с	T1/Тр
1	0,100	0,074	0,050	0,074	1,000
2	0,119	0,062	0,086	0,086	0,854
4	0,114	0,080	0,067	0,080	0,927
8	0,139	0,088	0,063	0,088	0,840
16	0,151	0,110	0,109	0,110	0,673
32	0,201	0,145	0,153	0,153	0,481

Лучшее время в этой короткой серии получено при $p=1$, а $p=32$ увеличило накладные расходы. Это не означает, что один раздел оптимален для больших данных: используются только два vCPU, небольшая вычислительная нагрузка и прогретая JVM. Серия не меняет число физических узлов.

Результаты приведены в таблице 9.

Таблица 9 – Отдельный опыт числа shuffle partitions

q	Группы	Время с
2	27	0,295
4	27	0,188
8	27	0,211
16	27	0,227
32	27	0,332

Во всех q получено 27 штатов. Это дополнительные одиночные измерения, а не серия медиан; после них shuffle.partitions возвращён к 8.

Результаты приведены в таблице 10.

Таблица 10 – Число выходных Parquet файлов

Партиции	Файлы	Объём МиБ	Средний КиБ
1	1	6,456	6 611,425
4	4	7,464	1 910,897
16	16	8,756	560,407
64	64	10,301	164,824

При росте p от 1 до 64 число part-файлов выросло от 1 до 64, общий объём — от 6,456 до 10,301 МиБ. Малые файлы имеют дополнительные метаданные и хуже используют сжатие. Учебный `coalesce(1)` создал один файл, но для больших фактов такой способ ограничивает запись одной задачей.

На рисунке 5 показан результат выполнения на стенде.

```

TIABD PRACTICE 4 | BENCHMARK
p      t1      t2      t3      median  speedup
1      0.100   0.074   0.050   0.074    1.000
2      0.119   0.062   0.086   0.086    0.854
4      0.114   0.080   0.067   0.080    0.927
8      0.139   0.088   0.063   0.088    0.840
16     0.151   0.110   0.109   0.110    0.673
32     0.201   0.145   0.153   0.153    0.481
files   MiB     mean KiB
1       6.456   6611.425
4       7.464   1910.897
16      8.756   560.407
64     10.301   164.824
student@tiabd-pr2:~/spark-lab$

```

Рисунок 5 – Реальные времена повторений и измеренные размеры part файлов

В дополнительном опыте `AQE=true` с исходными 32 shuffle partitions итоговая агрегация дала 27 групп и 1 выходную партицию. В финальном плане появился `AQEShuffleRead` с `coalesced`; затем базовая конфигурация восстановлена.

9 Журнал Spark UI

Результаты приведены в таблице 11.

Таблица 11 – Счётчики измеренных действий UI

Опыт	Jobs	Stages	Tasks	Read КиБ	Write КиБ
GroupBy	1	2	10	262,96	262,96
Repartition 4	1	4	23	172,58	172,58
Coalesce 4	1	3	7	151,84	151,84
SMJ	1	4	12	3 825,50	3 825,50
BHJ	2	3	4	0,12	0,12

Опыт	Jobs	Stages	Tasks	Read КиБ	Write КиБ
Материализация	3	4	5	0,12	0,12
Кэш действие	1	3	11	6,29	6,29

Счётчики получены из REST API живого приложения и сверены с UI. Stages учитываются без повторного суммирования внутри одного измерения. Последующая сортировка самостоятельного агрегата может повторно читать готовый shuffle, поэтому Read и Write не обязаны совпадать. Количество Jobs также не равно числу строк кода.

10 Самостоятельный вариант В

Прочитаны 112 650 позиций в двух входных партициях и 3 095 продавцов в одной. Select, filter(price>0) и withColumn(gross) не создали Jobs; count создал Job 97. JOIN по seller_id обеими стратегиями сохранил 112 650 строк.

Результаты приведены в таблице 12.

Таблица 12 – Штаты продавцов по gross

Штат	Позиции	Цена с доставкой
SP	80 342	10 235 883,88
PR	8 671	1 458 900,73
MG	8 827	1 224 159,80
RJ	4 818	937 814,12
SC	4 075	738 973,13

Группировка содержит Exchange по seller_state. Измеренный агрегат с сортировкой дал 3 Jobs, 27 выполненных Tasks, 5,20 КиБ Shuffle Read и 3,39 КиБ Write. Это включает сортировку, а не только HashAggregate.

Результаты приведены в таблице 13.

Таблица 13 – Контроль самостоятельного repartition coalesce

Способ	Минимум	Максимум	Среднее
repartition	28 161	28 165	28 162,50
coalesce	5 348	88 381	28 162,50

Результаты приведены в таблице 14.

Таблица 14 – JOIN варианта В

Стратегия	Время с	Stages	Tasks	Shuffle КиБ
SMJ	0,199	4	12	1 337,59
BHJ	0,084	3	4	0,12

Два действия без кэша заняли 0,293 с. Материализация заняла 0,155 с, повторные действия — 0,160 с; с учётом материализации 0,314 с. Кэш опять ускорил повторное использование, но не окупился за два действия.

Результаты приведены в таблице 15.

Таблица 15 – Три повторения для p варианта В

p	t1 с	t2 с	t3 с	Медиана с
1	0,102	0,112	0,118	0,112
2	0,134	0,124	0,139	0,134
4	0,132	0,117	0,121	0,121
8	0,157	0,154	0,167	0,157

Результаты приведены в таблице 16.

Таблица 16 – Запись агрегата варианта В

Партиции	Файлы	Объём МиБ
1	1	0,0013
4	4	0,0041

Каждому p предшествовал прогревочный запуск, затем использована медиана трёх повторов. Итог по seller_state дополнительно сохранён в exports/pr4_variant_parquet. Минимум два размера записи сопоставлены с фактическим числом файлов.

На рисунке 6 показан результат выполнения на стенде.

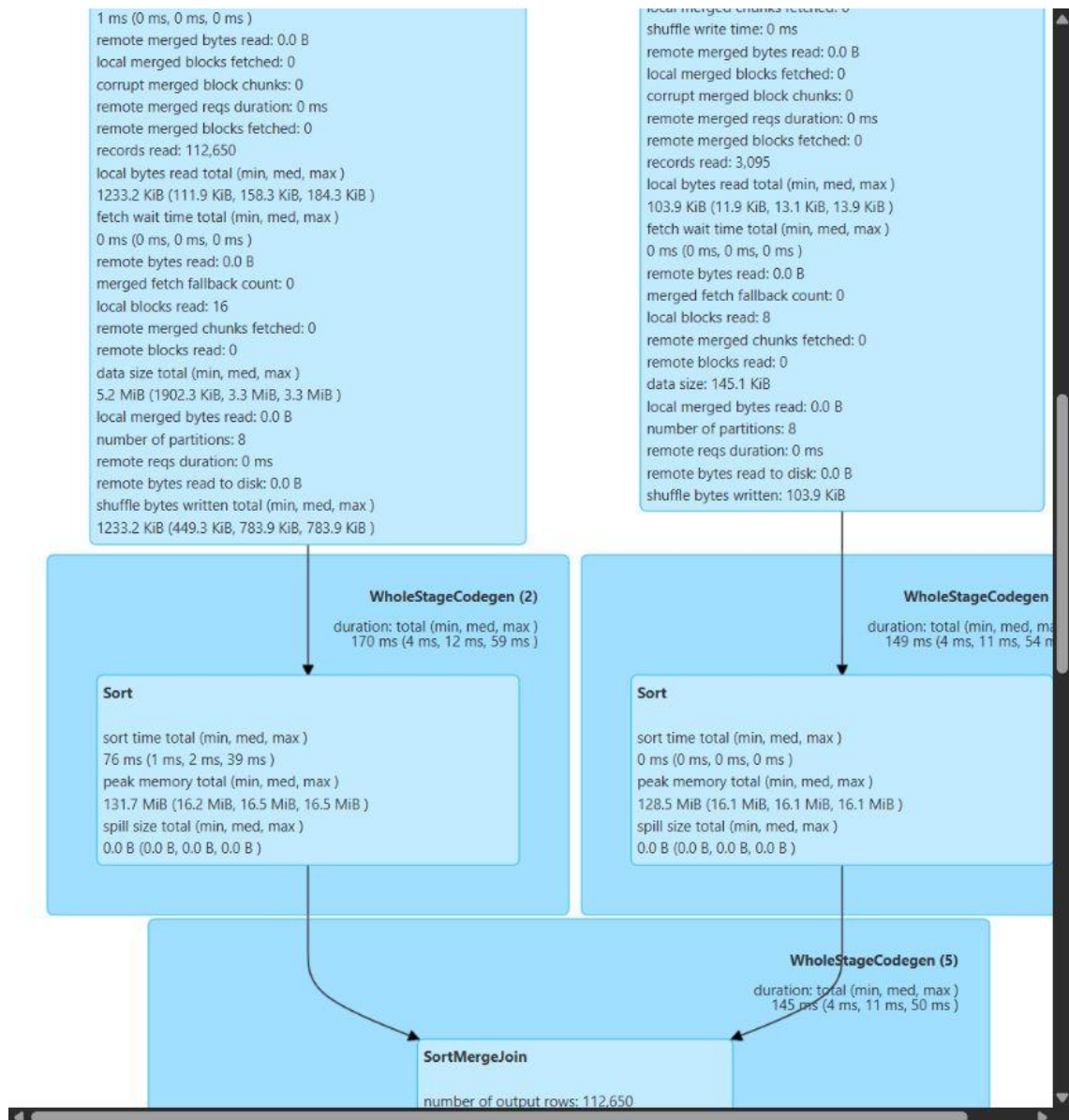


Рисунок 6 – SQL DataFrame план самостоятельного SMJ по seller_id

На рисунке 7 показан результат выполнения на стенде.


3.5.7

[Jobs](#)
[Stages](#)
[Storage](#)
[Environment](#)
[Executors](#)
[SQL / DataFrame](#)

TIABD_Practice_4_Albakhtin
application UI

Details for Job 98

Status: SUCCEEDED
Submitted: 2026/10/05 03:15:28
Duration: 0.1 s
Associated SQL Query: [73](#)
Job Group: P4_B_SMJ
Completed Stages: 4

[▶ Event Timeline](#)
[▶ DAG Visualization](#)

▼ **Completed Stages (4)**

Page:
1 Pages. Jump to . Show items in a page.

Stage Id ▼	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
217	P4_B_SMJ count at <unknown>:0 +details	2026/10/05 03:15:28	1 ms	1/1			472.0 B	
216	P4_B_SMJ count at <unknown>:0 +details	2026/10/05 03:15:28	0.1 s	8/8			1337.1 KiB	472.0 B
215	P4_B_SMJ count at <unknown>:0 +details	2026/10/05 03:15:28	26 ms	2/2	941.8 KiB			1233.2 KiB
214	P4_B_SMJ count at <unknown>:0 +details	2026/10/05 03:15:28	10 ms	1/1	128.7 KiB			103.9 KiB

Page:
1 Pages. Jump to . Show items in a page.

Рисунок 7 – Завершённые стадии Job 98: Tasks и shuffle самостоятельного варианта В

На рисунке 8 показан результат выполнения на стенде.

 3.5.7 Jobs Stages Storage Environment Executors SQL / DataFrame TIABD_Practice_4_Albakhtin application UI						
Storage						
▼ RDDs						
ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
955	*(2) Project [seller_id#206, price#208, freight_value#209, gross#2312, seller_zip_code_prefix#236, seller_city#237, seller_state#238] +- *(2) BroadcastHashJoin [seller_id#206, [seller_id#235], Inner, BuildRight, false :- *(2) Project [seller_id#206, price#208, freight_value#209, (price#208 + freight_value#209) AS gross#2312] : +- *(2) Filter ((isnotnull(price#208) AND (price#208 > 0.0)) AND isnotnull(seller_id#206)) : +- *(2) ColumnarToRow : +- FileScan parquet [seller_id#206,price#208,freight_value#209] Batched: true, DataFilters: [isnotnull(price#208), (price#208 > 0.0), isnotnull(seller_id#206)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[file:/home/student/spark-lab/data/spark_input_parquet/order_items], PartitionFilters: [], PushedFilters: [IsNotNull(price), GreaterThan(price,0.0), IsNotNull(seller_id)], ReadSchema: struct<seller_id:string,price:double,freight_value:double> +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [pl...	Disk Memory Serialized 1x Replicated	2	100.00%	2.6 MiB	0.0 B

Рисунок 8 – Materialized Storage самостоятельного варианта В

Самостоятельные выводы по варианту В

1. Связка order_items–sellers содержит 112 650 позиций. LEFT JOIN сохраняет их все, поэтому справочник покрывает фактовую таблицу; группировка по seller_state описывает штат продавца.
2. Цепочка преобразований до action не создала Jobs. count() запустил вычисление; результат и журнал заданий согласуются с ленивой моделью исполнения.
3. Repartition и coalesce дают различную структуру физического плана. Наличие Exchange показывает перераспределение; одинаковое число разделов не доказывает одинаковой нагрузки.

4. SMJ и BNJ вернули одинаковые 112 650 позиций. Broadcast небольшого справочника sellers изменил структуру стадий и обмен данными, сохранив смысл запроса.

5. Persist ускорил повторные действия, однако стоимость первоначальной материализации не окупилась за два действия в этой серии. Кэш полезен после оценки всей стоимости повторного использования.

6. Серия по четырём значениям p включает по три повторения. Медианы характеризуют этот локальный стенд с двумя vCPU; большее число разделов не гарантировало ускорения.

7. Запись с одним и четырьмя разделами создала соответственно один и четыре part-файла. Итог мал, поэтому накладные расходы формата существенно влияют на средний размер файла.

Ограничения самостоятельного опыта: один локальный виртуальный узел и короткие операции; влияние прогрева JVM, файлового кэша и фоновой нагрузки. Измерения нельзя переносить на физический кластер без нового эксперимента.

11 Выводы и ограничения

1. Цепочки transformations не создали Jobs; action запустил выполнение. При count оптимизатор вправе удалить неиспользуемые признаки.

2. Количество входных разделов и shuffle partitions различается: два входных раздела geolocation не означают две задачи каждой последующей стадии.

3. GroupBy seller_id создал Exchange и переход от двух к восьми задачам, что подтверждается планом и UI.

4. Repartition(4) выровнял geolocation почти идеально, а coalesce(4) сохранил сильный перекоп и имел меньшую цену данного count.

5. SMJ и BNJ сохранили одинаковые факты; BNJ исключил shuffle по ключу большой стороны и был быстрее в прогретой серии этого стенда.

6. Persist ускорил повторные действия, но материализация не окупилась за два действия ни в основной, ни в самостоятельной серии.

7. Для короткого benchmark $p=1$ оказался быстрее увеличенных p ; рост числа разделов увеличивает планирование и не гарантирует ускорение.

8. Увеличение выходных разделов создало больше файлов и увеличило общий размер Parquet. AQE смог объединить 32 shuffle partitions в одну выходную.

9. В варианте B справочник sellers покрывает все факты; смена стратегии JOIN изменила стадии и обмен, но не число строк.

10. В варианте B median зависела от p , а запись с 1 и 4 разделами дала соответствующие числа файлов; результаты подтверждены отдельными UI-фрагментами.

Ограничения: local[*] в VM с двумя vCPU не моделирует сеть и отказоустойчивость физического кластера. Измерения длительностью доли секунды зависят от JIT, прогрева, файлового кэша, нагрузки хоста и фоновых служб; три повтора и медиана уменьшают, но не устраняют эту неопределённость. Первые одиночные замеры JOIN, q и persist используются вместе с планами, а не как универсальные оценки производительности.

Ответы на контрольные вопросы

1. Чем transformation отличается от action с точки зрения момента фактического выполнения?

Transformation описывает новый план без обязательного вычисления. Action требует результат и запускает необходимые стадии выполнения.

2. Что означает lazy evaluation и почему замер времени вокруг withColumn() без Action некорректен?

Вычисления откладываются до action. Время withColumn без action измеряет построение выражения, а не обработку строк; для проверки вычисляемого поля нужен action, использующий его значение.

3. Как связаны Job, Stage и Task?

Job представляет выполнение действия, Stage объединяет задачи без необходимого промежуточного shuffle, Task обычно обрабатывает одну партицию стадии. Один action может породить несколько Jobs.

4. Почему число Tasks в разных Stages одного Job может отличаться?

До shuffle используются входные партиции, после него — число разделов нового распределения. Финальная глобальная агрегация может выполняться одной задачей.

5. Что такое partition и чем он отличается от физического CPU core?

Partition является частью данных, CPU core — аппаратным ресурсом. Один core последовательно выполняет задачи нескольких партиций; партиция не закреплена за ядром навсегда.

6. Почему local[*] сохраняет модель распределённого выполнения, хотя используется один компьютер?

Local[*] сохраняет планирование стадий, задачи и shuffle, но выполняет их потоками одной JVM. Сетевую масштабируемость такой запуск не проверяет.

7. Что называют narrow и wide transformation? Приведите примеры.

Narrow-операции используют ограниченное число входных разделов: filter, select, простые withColumn. Wide-операции перераспределяют данные: groupBy, repartition и многие JOIN.

8. Что такое shuffle и почему он обычно дороже локального filter()/select()?

Shuffle перераспределяет записи между разделами с записью и чтением промежуточных данных. Filter и select обычно работают локально; в кластере к цене shuffle добавляется сетевой обмен.

9. Как оператор Exchange в physical plan связан с shuffle?

Exchange обозначает изменение распределения данных. В зависимости от вида это shuffle либо broadcast, поэтому следует читать тип оператора, а не только его название.

10. В чём различие repartition() и coalesce()?

Repartition может увеличить или уменьшить число разделов и обычно выполняет shuffle. Coalesce преимущественно уменьшает число разделов объединением существующих без нового полного обмена.

11. Почему coalesce() может сохранить неравномерные размеры partitions?

Объединение сохраняет расположение исходных строк. Если крупные исходные партии попадут в одну группу, дисбаланс останется или усилится.

12. Что такое data skew и как он проявляется на geolocation_state?

Data skew — неравномерность объёма данных по ключам или партициям. SP содержит около 40,42% geolocation, поэтому hash-разбиение по штату может перегрузить отдельные задачи.

13. Что задаёт spark.sql.shuffle.partitions и почему это не то же самое, что число partitions исходного Parquet?

Параметр задаёт стандартное число shuffle partitions для SQL-операций. Входные Parquet partitions зависят от файлов, их размеров и параметров чтения; на стенде geolocation читается в двух разделах.

14. Какие physical operators необходимо уметь распознавать в explain("formatted")?

Нужно различать Scan, Filter, Project, Exchange, HashAggregate, Sort, SortMergeJoin, BroadcastExchange и BroadcastHashJoin. При persist полезны InMemoryTableScan, при AQE — AdaptiveSparkPlan и AQEShuffleRead.

15. Почему Sort Merge Join часто содержит Exchange и Sort с обеих сторон?

Обе стороны должны быть распределены по ключу соединения и отсортированы внутри разделов. Уже подходящая раскладка может устранить часть этих действий.

16. При каком условии Broadcast Hash Join может быть выгоден?

Если малая сторона помещается в память каждого исполнителя, её broadcast позволяет соединять большую сторону локально без shuffle по ключу JOIN.

17. Почему большую таблицу нельзя безусловно broadcast-ить?

Broadcast создаёт копию таблицы у исполнителей и hash-структуру. Слишком большой объём может исчерпать память и увеличить задержку передачи или привести к тайм-ауту.

18. Почему одинаковое число строк результата важно при сравнении двух JOIN-стратегий?

Это базовая проверка логической эквивалентности: нельзя сравнивать скорость запросов, возвращающих разные факты. Дополнительно проверяются ключи и уникальность справочника.

19. Что происходит при первом Action после persist()?

Первое действие вычисляет lineage и заполняет зарегистрированный кэш. Сам persist только объявляет сохранение; стоимость материализации должна входить в сравнение.

20. Почему cache() может ухудшить одноразовую задачу?

Одноразовая задача получает дополнительные расходы на запись и хранение кэша без последующего повторного использования. На стенде даже двух действий оказалось недостаточно для окупаемости.

21. Для чего нужен unpersist()?

Unpersist освобождает сохранённые блоки. Blocking=True позволяет дожидаться освобождения перед следующей серией экспериментов.

22. Почему единичный benchmark-run недостаточен и зачем использовать медиану?

Прогрев JVM, JIT, дисковый кэш и фоновые процессы меняют время отдельных запусков. Медиана нескольких повторов менее чувствительна к отдельному выбросу, но не заменяет контроль условий.

23. Какие факторы делают абсолютное время Spark-задачи несопоставимым между разными компьютерами?

Влияют CPU, память, виртуализация, диски, версии Java и Spark, параметры сессии и фоновая нагрузка. Сравнивать следует варианты одного контролируемого стенда.

24. Почему слишком большое число partitions увеличивает overhead?

Возникает больше событий планировщика, сериализации, служебных объектов и мелких задач. Их расходы могут превысить выигрыш параллелизма.

25. Как связаны output partitions, Tasks записи и число part-файлов?

Выходная задача обычно записывает файл своей партиции. Пустые разделы, разделение записи по каталогам и настройки формата могут нарушить точное равенство; число файлов нужно измерять.

26. Почему coalesce(1) не является универсальным способом борьбы с small files?

Один раздел устраняет параллелизм записи и концентрирует данные в одном потоке. Размер набора и дальнейшее чтение важнее самого желания иметь один файл.

27. Что меняет Adaptive Query Execution и почему основные опыты этой работы выполняются с AQE=false?

AQE использует фактическую статистику, объединяет мелкие shuffle partitions и может менять стратегию JOIN. AQE=false фиксирует условия основных опытов и упрощает сравнение.

28. Какие вкладки Spark UI наиболее полезны для анализа groupBy, JOIN и cache?

Jobs связывает действие с выполнением; Stages показывает задачи и shuffle; SQL/DataFrame раскрывает физический план; Storage подтверждает материализацию и освобождение кэша.

29. В чём различие между логическим выводом о данных и экспериментальным выводом о вычислительной системе?

Логический вывод описывает данные, например выручку штата. Вычислительный вывод описывает цену конкретного плана при фиксированных данных и конфигурации.

30. Как сформулировать корректный вывод о производительности так, чтобы не превратить локальный результат в универсальное правило?

Следует назвать данные, оборудование и параметры, изменённый фактор, число повторов, медиану и подтверждение в плане/UI. Вывод ограничивается этими условиями, а не распространяется на все кластеры.

Источники

1. Технологии и инструментарий анализа больших данных. Практика №4. Предоставленная методичка.

2. Восстановленные таблицы Olist второй практики, source_manifest.json.

3. Apache Spark 3.5.7. Monitoring and Instrumentation.
<https://spark.apache.org/docs/3.5.7/monitoring.html>

4. Apache Spark 3.5.7. Tuning.
<https://spark.apache.org/docs/3.5.7/tuning.html>